# PROJECT AND TEAM INFORMATION

## Project Title

| |
|---|
| *Disaster Evacuation Management System* |

## Student / Team Information

| | |
|---|---|
| *Team Name:*<br>*Team-DSCPP-III-2026-T189* | *RescueNet* |
| **Team member 1 (Team Lead)** | *Koli, Piyush –*<br>*2512510030*<br>*2513510030@geu.ac.in*<br>  |
| **Team member 2** | *Rawat, Ayushi –*<br>*2510380094*<br>*2510380094@geu.ac.in*<br>  |
| **Team member 3** | *Bohra, Jatin –*<br>*2510370140*<br>*2510370140@geu.ac.in*<br>  |
| **Team member 4**<br>*(Last Name, name: student ID: email, picture):* | *Last Name, Name –*<br>*Student ID*<br>*email@example.com*<br>`member4.png` |

# PROPOSAL DESCRIPTION

## Motivation

*Floods are among the frequent and widely felt disasters in India. The hardest part of responding to a flood is rarely the lack of maps. Decision-making must happen when conditions keep changing. During an evacuation many problems happen once. Dozens of evacuation requests arrive from areas. Shelters have limited capacity that shrinks steadily. Roads become impassable without warning as water levels rise.*

*Existing navigation applications do not address this issue. They answer one question. The shortest path between two fixed points on a road network that is assumed to be static and fully available. An evacuation coordinator faces more questions. Who should be moved first? Which shelter can actually accommodate the evacuees? Which route is still usable now? A route to a shelter that is already full is useless. The nearest shelter is not always suitable.*

*In practice this coordination is still done manually over phone and radio. This leads to delays, duplicated dispatch of rescue teams and shelters being overcrowded while others remain empty. Information about blocked roads comes largely from citizens and field volunteers. Such reports cannot be applied blindly. A single mistaken or outdated report if acted upon automatically could divert evacuees away from a road that is perfectly usable.*

*This project is motivated by the need for a system that treats evacuation as a resource-allocation and scheduling problem than a navigation problem. The project aims to build a simulation that prioritises evacuation requests. The simulation allocates shelters by remaining capacity. It computes routes, over the available road network. The network is updated after a reported road condition has been verified by an operator.*

## State of the Art / Current solution

*At the moment the problem is handled by three types of tools that do not work well together.*

*Navigation platforms like Google Maps and OpenStreetMap-based routers use algorithms such as Dijkstra and A* to find the fastest routes. These tools are effective for navigation.. They assume the destination is already known. They treat the road network as fixed and do not consider shelter capacity. They also do not know which requests are more urgent than others.*

*In disaster management decisions at the district level still rely heavily on control rooms. Officials get requests by phone and radio. Teams are sent out by hand. Shelter occupancy is tracked using paper logs or spreadsheets. There are platforms like Sahana Eden that help manage shelters and resources.. The decisions about who gets where are made by people. These systems are not connected to live routing.*

*Then there is crowdsourced crisis mapping. Tools like Ushahidi and social media during events such as the Chennai and Kerala floods help collect reports about flooded roads. These reports are shown on maps.. The data does not go into any routing system. Verifying the reports is a manual task.*

*Because of this prioritization, shelter allocation, road condition verification and route calculation happen in places. They are handled by tools or, by people. There is no system that brings all these steps together. The process is not integrated.*

## Project Goals and Milestones

*General goal. To. Implement a simulation-based Flood and Disaster Evacuation Management System in C++ that brings together request prioritisation, shelter allocation, road-condition verification and dynamic route computation in one decision process. All core data structures such as adjacency list, min-heap and hash table will be written by hand of using the STL so the project shows how to build data structures and model objects.*

*Specific objectives*

*Model a city road network as a weighted graph. Store it in an adjacency list. Rank evacuation requests by urgency of by arrival time. Assign shelters based on how much capacity is left and on route cost not just how close they are. Refresh the network only after a road report has been checked by an operator and then recalculate the affected routes.*

*Milestones*

*M1. Domain model.. Bundle the core classes: Location, Road, Shelter, EvacuationRequest, Road-Report, Route. Build a console menu.*

*M2. Graph layer. Create a custom adjacency-list structure; load a city with about 10–12 locations and their roads; show the network and the status of each road.*

*M3. Routing engine. Build a binary min-heap; run Dijkstra only on roads whose status is OPEN; handle the case when no route is available.*

*M4. Evacuation logic. Use a priority queue for requests; allocate shelters by how much capacity remains; update shelter occupancy when an evacuation succeeds.*

*M5. Reporting. Keep a queue of reports use a hash table for O(1) lookup of reports and roads by ID let the operator verify or reject update the edge status and recalculate routes.*

*M6. Integration and testing. Run disaster scenarios gather statistics by sorting create documentation and write the final report.*

## Project Approach

*The Flood & Disaster Evacuation Management System will be designed as a two-layer application. The Flood  Disaster Evacuation Management System separates the data and algorithm engine from the application logic. Both layers are written entirely in C++ so that the Flood  Disaster Evacuation Management System is supported nativly on every side. The core engine treats the road network as a graph. The graph is stored with an adjasency list and which is eficient for sparse real-world road networks. Dijkstra's algorithm is is used to find the minimum-cost route from an area to a candidate shelter. the flood and disaster Evacuation Management System handles evacuation requests and with a priority queue. Critical requests—based on danger level and group size or vulnerability—are processed before lower-priority ones. Road-condition reports are handled through a queue and processed in the order they arrive. a hash map gives lookup of shelters and requests and road reports by I D. The Flood and Disaster Evacuation Management System's application layer models real-world entities as C++ classes: Location, Road, Shelter and EvacuationRequest, RoadReport andOperator and Route. Each class keeps its state private. For example a Shelter's capacity and occupancy are private.*

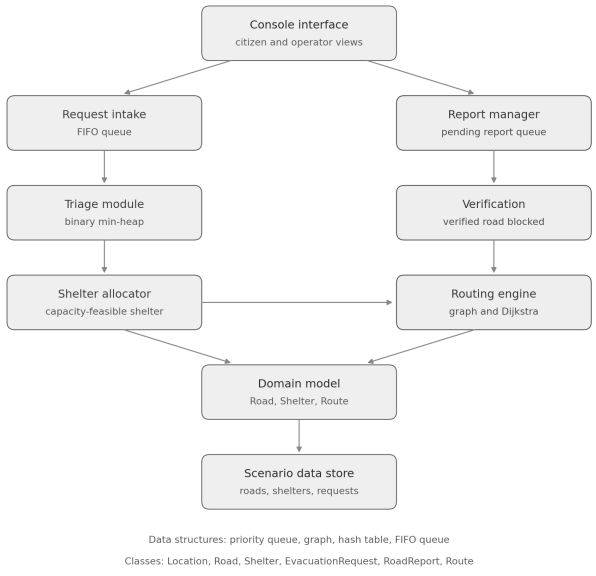
*they can only be changed through the method acceptEvacuees().*

*The RoadReport class models a verification workflow: pending→ verified or rejected . The graph only updates when an operator confirms that a citizen report is accurate.*

*For interaction the Flood and Disaster Evacuation Management System will start as a command-line interface. The goal is to show DSA and OOP usage and not a polished UI. If time allows a simple text-based dashboard could be added later to show pending requests, shelter occupancy and blocked roads.*

*The Flood & Disaster Evacuation Management System's build order is bottom-up. First build the graph, adjacency list and Dijkstra implementation. Next add the priority queue and hash map layers. Then create the OOP class hierarchy that wraps these structures. Add the report-verification workflow. Finally test the cycle of evacuate → block → reroute.*

## System Architecture (High Level Diagram)



## Project Outcome / Deliverables

*The main goal is a working console-based simulation of flood evacuation management written in C++ that takes a disaster scenario prioritises evacuation requests assigns each request to a shelter that has remaining space calculates a route over the roads that are open at the moment and recalculates that route whenever a verified report blocks a road.*

*Deliverables*

*- Source code. A C++ implementation that includes a custom graph built with an adjacency list, a binary min-heap, a hash table and a FIFO queue and it also contains the domain classes that are kept up to date on GitHub.*

*- Scenario data files. A simulated city network that lists locations, roads, shelters and evacuation requests in plain text and it can be edited without recompiling the program.*

*- Executable application. A menu-driven interface that offers a view for citizens and a view for operators.*

*- Test scenario suite. Documented cases that show expected outcomes such as normal evacuation, a shelter that is, at capacity a road that is blocked during evacuation a rejected report and a situation where no route is available.*

*- Project report. A document that explains the design rationale shows a class diagram justifies the chosen algorithms and analyzes their complexity.*

*- Demonstration. A walkthrough that shows the entire cycle from submitting a report to verifying it and then rerouting evacuees.*

## Assumptions

*The system works in a disaster scenario. No sensor, satellite or traffic feed is used. Road travel times are. Never change because of traffic or water. Road status is simple. It is either OPEN or BLOCKED never partly open. Road condition reports are entered by hand. The operators check is the final decision. Each evacuation request moves as one group, to one shelter. The shelters remaining capacity limits admission. Request priority is set when the request is entered and one operator uses the system at a time.*

## References

*1. S. Kongsomsaksakul, C. Yang and A. Chen, "Shelter location-allocation model for flood evacuation planning," Journal of the Eastern Asia Society for Transportation Studies, vol. 6, 2005.*

*2. Sahana Software Foundation, "Open source disaster management solutions." https://sahanafoundation.org/*

*3. Ushahidi, "Crowdsourced crisis mapping platform." https://www.ushahidi.com/*

*4. National Disaster Management Authority (NDMA), Government of India, "Flood management guidelines." https://ndma.gov.in/*

*5. Sphere Association, "The Sphere Handbook: Humanitarian charter and minimum standards in humanitarian response," shelter and settlement standards. https://spherestandards.org/*

*6. Central Water Commission (CWC), Government of India, "Flood forecasting and warning services." https://cwc.gov.in/*